

codeHive ACADEMY

BUBBLE SORT

THE RISING ALGORITHM

Study Guide Version 2.0
2026 Computing Fundamentals

What is Bubble Sort?

Bubble Sort is an algorithm that sorts an array from the lowest value to the highest value.

The name "Bubble" refers to the way that the highest values "bubble up" to the end of the array with each iteration, similar to air bubbles in water.

How it works:

- Iterate through the array, one value at a time.
- Compare the current value with the **next** value.
- If the current value is **higher** than the next one, **swap** them.
- Repeat this process for the entire array multiple times.

Manual Trace: One Pass

Let's look at an unsorted array: [7, 12, 9, 11, 3]

Step 1: Compare 7 and 12. $7 < 12$, No swap needed.



Step 2: Compare 12 and 9. $12 > 9$, **Swap!**



Step 3: Compare 12 and 11. $12 > 11$, **Swap!**



Step 4: Compare 12 and 3. $12 > 3$, **Swap!**



codehive

What just happened?

Observation:

After the first run-through, the **highest value (12)** has bubbled up to the final position. While 12 is correctly placed, the rest of the array (7, 9, 11, 3) is still unsorted.

The Repetition:

To fully sort the array, the algorithm must run through it again and again. Each time, the next highest value reaches its correct destination.

- Pass 2 will lock 11 into place.
- Pass 3 will lock 9 into place.
- Pass 4 will lock 7 into place.

For an array of **N** elements, the outer loop must ideally run **N-1** times.

Implementation: Python

We use two loops: an **outer loop** to repeat the passes, and an **inner loop** to compare neighbors.

```
# Standard Bubble Sort
my_array = [64, 34, 25, 12, 22, 11, 90, 5]

n = len(my_array)
for i in range(n-1):
    for j in range(n-i-1):
        if my_array[j] > my_array[j+1]:
            # Perform the swap
            my_array[j], my_array[j+1] = my_array[j+1], my_array[j]

print("Sorted array:", my_array)
```

Note: We use $n-i-1$ in the inner loop because the last i elements are already sorted and don't need to be checked again.

The Optimization: "Early Exit"

If an array is already sorted, we shouldn't keep running. We can use a swapped flag to stop early.

```
# Optimized Bubble Sort
for i in range(n-1):
    swapped = False
    for j in range(n-i-1):
        if my_array[j] > my_array[j+1]:
            my_array[j], my_array[j+1] = my_array[j+1], my_array[j]
            swapped = True

    if not swapped:
        break # Done sorting!
```

Efficiency Gain: If no swaps happen during a full pass, the array is guaranteed to be sorted. This prevents unnecessary work on already sorted data.

Complexity Analysis

Understanding the "cost" of your algorithm is vital for working with large datasets.

Scenario	Time Complexity	Reason
Worst Case	$O(N^2)$	Array is in reverse order.
Best Case (Optimized)	$O(N)$	Array is already sorted.
Average Case	$O(N^2)$	General unsorted data.

Big O: $O(n^2)$

Quadratic time complexity means performance drops quickly as N grows.

Test Your Knowledge

Challenge: You have the array [7, 14, 11, 8, 9].

How does the array look after the **FIRST** complete pass through the algorithm?

[__, __, __, __, __]

Hint: The largest value (14) will move to the end.

© 2026 codeHive Academy | Empowerment through Logic

Master these basics to unlock advanced sorting: QuickSort and MergeSort await.